

RevenueOS

AI-powered revenue recovery and orchestration platform

Razorpay AI Buildathon 2026 — Track 03

The Problem

Most payment recovery systems operate on a dangerously simple assumption: if a charge failed, retry it. This approach is cheap to automate — but expensive in practice.

Blind Retries

Retry logic ignores customer value, churn risk, and instrument viability — treating every failure as identical.

FIFO Queues

First-in, first-out queues replace expected-rupee prioritization — high-value recoveries wait behind low-probability ones.

No Intelligence

Systems fire actions without asking whether the event is worth pursuing or what the safest next step actually is.

Failure Modes

Blind automation compounds the original failure. Each of these failure modes erodes revenue, trust, and customer relationships simultaneously.

1 Retrying Dead Instruments

Expired cards and blocked accounts can never succeed — yet systems keep trying.

3 Customer Fatigue

Repeated failed attempts erode trust and accelerate churn — the opposite of recovery.

2 Wasted Contact Attempts

Low-value, low-probability events consume the same resources as high-ERV recoveries.

4 No Stopping Rule

Without a termination condition, intervention continues even when it isn't justified.

4

ways blind retry logic destroys value

Every failure mode is a compounding cost — in rupees, in relationships, and in recoverable revenue left on the table.

The Question We Answer

Which revenue is realistically recoverable, why should we pursue it, and what's the safest and most valuable next action?



How It Works

RevenueOS runs a structured pipeline — from raw event ingestion to intelligent action orchestration — powered by ML models and a LangGraph agent loop.



Validation

Canonical event context built from payment, attempt, and customer features



Prediction

Recovery & churn prediction via sklearn models



Risk Engine

ERV + priority score computed; recoverability class assigned



Orchestration

LangGraph agent selects and executes the optimal action

Key Distinctions

RevenueOS is precise about what it claims — and what it doesn't. These distinctions are what separate intelligent orchestration from naive automation.

Revenue at Risk ≠ Predicted ERV ≠ Actual Recovered

Three distinct quantities — conflating them leads to overconfident systems and misleading dashboards.



Not Every Event Deserves a Recovery Attempt

The system explicitly classifies events as NOT_RECOVERABLE or ACTION_BLOCKED and stops — by design.



LLM Doesn't Control Guardrails

Policy checks run in deterministic code — not in the LLM. The agent cannot override compliance rules.

Agent Architecture



Node Responsibilities

Supervisor

Routes to the next node or terminates the loop — the central decision-maker.

Investigator · Diagnosis · Customer Analyst · Strategist

Specialized reasoning nodes that build context, diagnose failure, profile the customer, and select strategy.

Guardrail

Policy enforcement in deterministic code — cooldown, max attempts, compliance. Not LLM-controlled.

Execution · Observer · Reflection

Action execution, outcome observation, and loop reflection to close the agentic cycle.

Recoverability Classes

Every failed payment event is classified into one of five recoverability states before any action is taken. This classification gates the entire downstream pipeline.

RECOVERABLE

Worth pursuing — high probability and sufficient ERV to justify action.

LOW_RECOVERABILITY

Limited potential — marginal cases where intervention cost may exceed return.

NOT_RECOVERABLE

Don't send another action — expired instruments, permanently blocked accounts.

ACTION_BLOCKED

Policy blocks — cooldown period active, maximum attempts reached.

NEEDS_REVIEW

Incomplete context — insufficient signal to classify; escalated for human review.

Supported Actions

Once a recoverability class is assigned and the priority score computed, the agent selects from a defined action set. Each action is matched to the event's risk profile — including an explicit **stop** action when intervention isn't warranted.

- ❏ The **Stop Recovery** action is a first-class citizen — not an edge case. Knowing when *not* to act is as valuable as knowing what action to take.



Retry Now / Retry Later

Immediate or scheduled retry based on failure reason and instrument viability.



Payment Link + Email Reminder

Generate a fresh payment link or send a targeted email nudge to the customer.



Method Update + Retention Offer

Prompt instrument update or issue a retention offer for high-LTV + high-churn events.

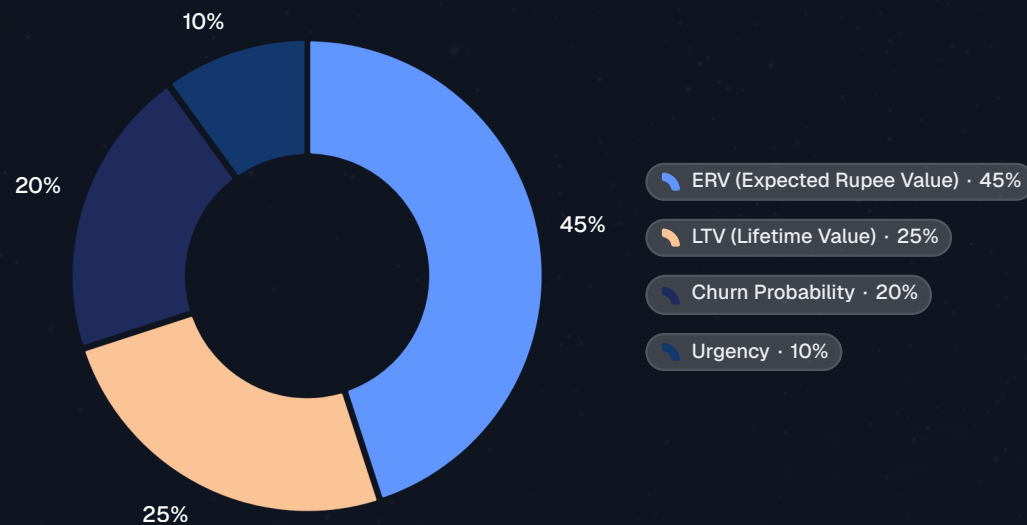


Stop Recovery

Explicit no-action — terminates the loop when intervention isn't justified.

Risk Scoring

Every event receives a **priority score (0–100)** computed from four weighted signals. This score drives queue ordering and action selection across the entire platform.



Recovery Model

P(recovery) derived from payment features, attempt history, failure reason, and customer attributes via sklearn.

Churn Model

P(churn) derived from engagement signals, customer tenure, and historical spend patterns via sklearn.

Priority Score

ERV 45% + LTV 25% + Churn 20% + Urgency 10% — a single 0–100 score that replaces FIFO ordering.